

PerturbGen JEPA Phase A — Architecture & Ablation Brief

Purpose: Self-contained handoff for evaluating freeze / unfreeze / fresh-encoder alternatives.

Date: 2026-08-04

Repo: Perturbgen (JEPA research path under perturbgen/)

1. Project goal

Replace (eventually) PerturbGen's **MaskGIT** trajectory backbone with a **cell-level JEPA**: predict future **cell latents** from a source cell, then later decode to gene programs / counts (Phase D+).

Phase A only trains the JEPA latent predictor and monitors representation quality / collapse.

This is **not** gene-block JEPA: one embedding per cell via mean-pool over gene tokens.

2. Data

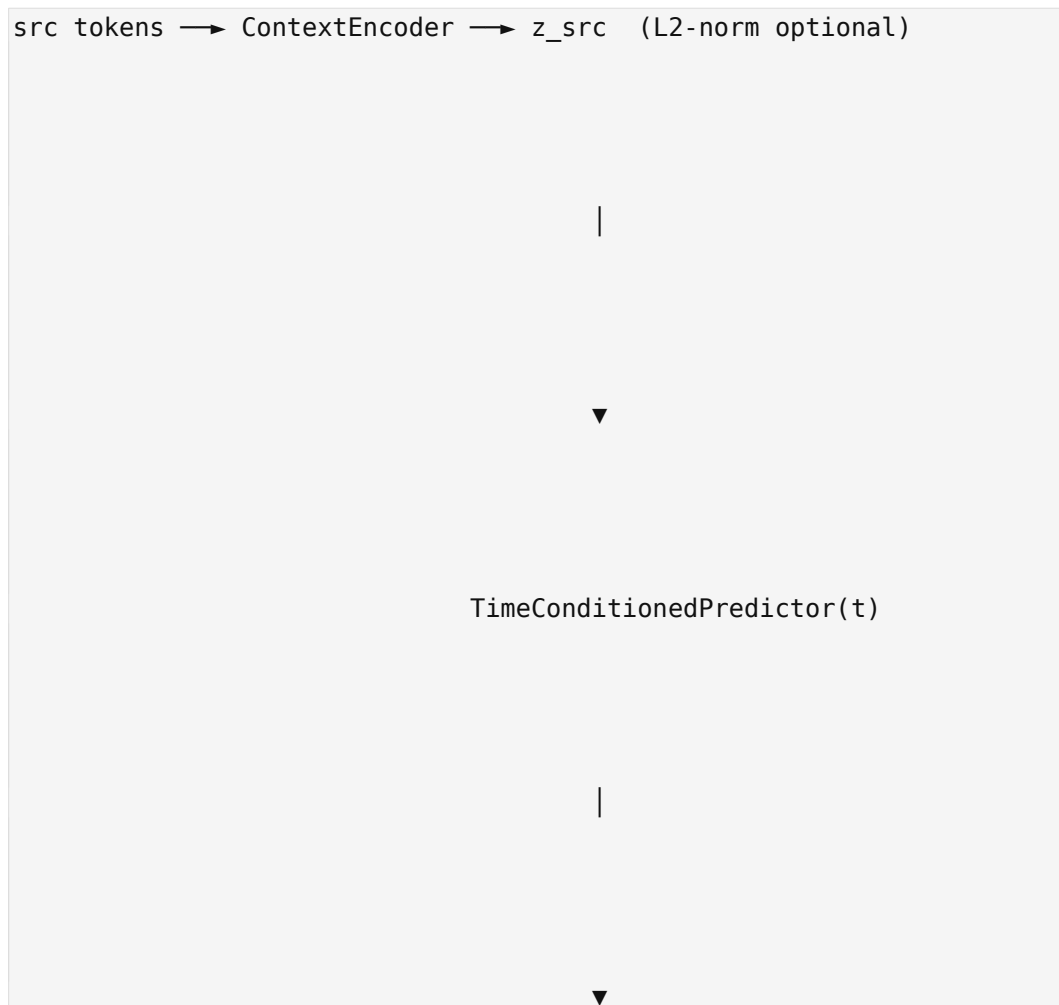
Item	Value
Dataset	LPS time-course, 2k HVG tokenized pairs
Total pairs	~148,107
Train / val / test	118,485 / 14,811 / 14,811
Source	Baseline / unstimulated cells (src_input_ids)

Item	Value
Target ts	Same pairing at LPS times t = 1, 2, 3

Dual ID spaces

- **Source:** global Geneformer / pretrain token IDs
- **Target:** local HVG remapped IDs
- **Bridge:** tokenid_to_rowid_2000_hvg.pkl
- **scmaskgit encoder:** remap target local → global
- **cell encoder:** remap source global → local (HVG vocab)

3. Main architecture (CellTrajectoryJEPA)



$z_{\text{hat}}(t) \leftarrow \text{loss vs } z_{\text{tgt}}(t)$

$\text{tgt tokens@t} \rightarrow \text{TargetEncoder (EMA, stop-grad)} \rightarrow z_{\text{tgt}}(t)$

3.1 Context encoder (online)

Maps gene-token sequence $\rightarrow$ token embeddings $\rightarrow$ **mean-pool** $\rightarrow$ $\text{cell_embedding} \in \mathbb{R}^d$.

Two backends:

**B
a
c
k
e
n
d**

**C
L
I**

**Initi
aliz
atio
n**

**W
i
d
t
h**

Pr
et
ra
in
e
d
M
as
k
Gl
T

-
-
j
e
p
a
-
e
n
c
o
d
e
r

s
c
m

Loa
d
full
12-
laye
r
scmo
scf
che
ckp
oint

d
=
7
6
8
,
8

h
e
a
d
s

**B
a
c
k
e
n
d**

**C
L
I**

**Initi
aliz
atio
n**

**W
i
d
t
h**

a
s
k
g
i
t

**B
a
c
k
e
n
d**

**C
L
I**

**Initi
aliz
atio
n**

**W
i
d
t
h**

**B
a
c
k
e
n
d**

**C
L
I**

**Initi
aliz
atio
n**

**W
i
d
t
h**

Fr
es
h
C
el
IE
n
c
o
d

-
-
j
e
p
a
-
e
n
c
o
d

Ran
dom
(Xav
ier
emb
eddi
ngs)

C
L
I

d
-
m
o
d
e
l

**B
a
c
k
e
n
d**

**C
L
I**

**Initi
aliz
atio
n**

**W
i
d
t
h**

/

n
u
m
-
h
e
a
d
s

/

n
u
m
-
l
a
y
e
r
s

/

d
-
f
f

er

e
r

c
e
l
l

SCMaskGITCellEncoder: load pretrained MaskGIT ckpt; encode with

MaskGIT positional encoding; run first N transformer blocks; mean-pool non-padding tokens.

CellEncoder: token emb $\times \sqrt{d} \rightarrow$ time + position sinusoidal $\rightarrow$ TransformerEncoder $\rightarrow$ mean-pool (exclude pad/mask).

Optional for cell only: warm-start **token embedding** from a MaskGIT ckpt (not the full encoder). For scmaskgit, the full encoder is already loaded; warm-start is a no-op.

3.2 Target encoder (EMA)

- Deep copy of the context encoder; all parameters requires_grad=False
- Updated each training batch:
 $\theta_{tgt} \leftarrow \tau \cdot \theta_{tgt} + (1 - \tau) \cdot \theta_{ctx}$ with $\tau = \text{ema_decay}$ (default **0.996**)
- Forward under torch.no_grad() / eval mode

3.3 Predictor

TimeConditionedPredictor:

- Time embedding for target time index t
- MLP on $[z_{src} \parallel t_{emb}] \rightarrow z_{hat}$ (GELU MLP, hidden = $4 \times d$)
- Always trained in Phase A

Times predicted jointly: **$t \in \{1, 2, 3\}$** from the **same** z_{src} .

3.4 Normalization

If normalize_latents=true (default): L2-normalize z_{src} , z_{tgt} , and z_{hat} before the loss.

3.5 Freeze flag (--freeze_jepa_encoder)

S
e
t
t
i
n
g

Effect

t

Context encoder frozen. Optimizer trains predictor

S
e
t
t
i
n
g

Effect

r only. EMA target stays a frozen copy of the (frozen)
u encoder. VICReg applied to **predictor outputs**
e **only**.

f
a Context encoder trainable (for scmaskgit: only the
l first N blocks that still have gradients). VICReg on
s **z_hat and z_src** (averaged).
e

Important: For the **fresh** (cell) alternative, the encoder must **not** be frozen. Freezing random weights and training only the predictor is not a meaningful from-scratch run.

4. Loss and monitors (Phase A)

Primary latent loss (--jepa_loss)

- **cosine** (main recipe): $\text{mean}(1 - \cos(z_{\text{hat}}, z_{\text{tgt}}))$
- Also available: mse, smooth_l1

Anti-collapse VICReg (optional)

$$L = L_{\text{pred}} + \lambda_{\text{var}} \cdot \text{Var} + \lambda_{\text{cov}} \cdot \text{Cov}$$

Main recipe: $\lambda_{\text{var}} = 1.0$, $\lambda_{\text{cov}} = 0.04$.

Default γ : $1/\sqrt{d}$ if latents are normalized, else 1.

Logged diagnostics

- jepa_pred_loss, jepa_loss (total), per-time jepa_loss_t{1,2,3}

- latent_cosine
 - collapse_mean_cosine (mean off-diagonal pairwise cosine of targets — **high $\approx$ collapsed**)
 - collapse_std_mean
 - Baselines: identity (z_{src} vs z_{tgt}) vs JEPa (z_{hat} vs z_{tgt})
 - beats_identity: fraction of examples where JEPa beats identity on cosine loss
-

5. Shared training recipe

Keep fixed across ablations unless noted:

Hyperparameter	Value
Global batch size	256 (4 GPUs $\rightarrow$ 64 / GPU)
Learning rate	1e-4
Weight decay	1e-4
Epochs	$\sim$ 30
GPUs	typically 0,1,2,7
Loss	cosine + VICReg (var=1, cov=0.04)
Distributed	LightningEnvironment + NCCL (not OpenMPI)
WandB	disabled on headless host

Launch script: docs/examples/run_train_jepa_sod2.sh

Outputs (sod2): .../T_perturb/res/jepa/checkpoints/ and .../logs/

Successful baseline already completed

Field	Value
Run ID	20260804_1120
Config	scmaskgit, 3 layers, unfrozen , cosine+VICReg, bs=256
Best epoch	~17
Val pred loss	~0.065
Val latent cosine	~0.935
Beats identity	~0.93
Yellow flag	val collapse_mean_cosine rose ~0.59 → ~0.85
Kept checkpoints	epoch 09 and 19 (prefer 19)

6. Alternatives to compare

Keep data, loss, VICReg, batch, LR, and prediction times fixed. Change only encoder backend and freeze.

Alternative A — Pretrained + unfrozen (unfz) — DONE (1120)

- `jepa_encoder=scmaskgit, jepa_encoder_layers=3, freeze_jepa_encoder=false`
- **Trains:** first 3 MaskGIT blocks (and related early params with grad) + predictor; EMA tracks online encoder
- **Hypothesis:** adapting pretrained features + predictor gives

best Phase A metrics

- **Risk:** representation drift / collapse (observed: high pairwise cosine on val)
- **Name tags:** enc_scmaskgit_L3_unfz_loss_cosine_vicv_...

Alternative B — Pretrained + frozen (fz)

- Same encoder load and early-exit 3L; freeze_jepa_encoder=true
- **Trains:** predictor only; encoder = fixed pretrained features; EMA $\approx$ constant copy of that encoder
- **Hypothesis:** if MaskGIT cell features already carry LPS-relevant structure, the predictor alone can map $z_{\text{src}} \rightarrow z_{\text{tgt}}(t)$
- **If much worse than A:** encoder adaptation is needed for this trajectory task
- **If $\approx$ A:** freeze is enough $\rightarrow$ cheaper, less collapse risk from encoder updates
- **Name tags:** enc_scmaskgit_L3_fz_...

Alternative C — Fresh encoder (cell, unfrozen)

- jepa_encoder=cell, freeze_jepa_encoder=false (**do not freeze**)
- Random CellEncoder (optionally no token-emb warm-start for a pure fresh run)
- **Trains:** full cell encoder + predictor + EMA
- **Hypothesis:** JEPA objective alone can learn useful cell trajectory latents without MaskGIT
- **Expect:** slower convergence, lower cosine early, possibly more collapse unless capacity / VICReg are tuned
- **Caveat:** CLI defaults for the cell path can be awkward ($d_{\text{model}}=768$ with default $d_{\text{ff}}=64$, $\text{num_layers}=6$). For a fair ablation, recommend e.g. $\text{num_layers}=3$, $d_{\text{ff}}=3072$ (or another matched capacity) so a weak FFN does not confound the comparison
- **Name tags:** enc_cell_unfz_...

7. Evaluation checklist (“does it work?”)

For each alternative, at **best validation epoch** (and last epoch):

Criterion	Good sign
val/	Decreases vs epoch 0; competitive

Criterion	Good sign
jepa_pred_loss	with 1120 (~0.065)
val/ latent_cosine	Increases toward ~0.9+
val/ beats_identity	$\gg 0.5$ (ideally ~0.9)
JEPA vs identity cosine-loss	JEPA lower
val/ collapse_mean _cosine	Prefer not climbing to ~0.85+; lower = more diverse cells
Per-time losses	t2 often easiest; check t1/t3 do not lag badly
Train/val gap	Mild overfit OK (1120 best ~17, slight drift by ~27)

Relative expectations (prior)

1. **A (unfz pretrained) $\geq$ B (fz) $\geq$ C (fresh)** on pred loss / cosine, if pretraining helps.
2. **B $\approx$ A** $\Rightarrow$ freeze is sufficient for Phase A.
3. **C $\ll$ A** $\Rightarrow$ MaskGIT initialization is doing real work; do not drop it for Phase B yet.
4. **C $\approx$ A** with healthy collapse $\Rightarrow$ JEPA can stand alone; MaskGIT optional later.

Downstream (roadmap, not Phase A)

Phase D JEPACountDecoder maps latents $\rightarrow$ counts; then compare to MaskGIT PerturbGen on expression / gene-program metrics.

8. Key code pointers

Component	Path
Backbone	perturbgen/Modules/jepa.py
Pretrained encoder	perturbgen/Modules/ jepa_scmaskgit.py
Train loop / loss / VICReg	perturbgen/Model/ jepa_trainer.py
Metrics	perturbgen/src/ jepa_metrics.py
CLI / DDP / checkpoint naming	perturbgen/train.py
Launch	docs/examples/ run_train_jepa_sod2.sh
Curves	docs/examples/ plot_jepa_curves.py

9. Infrastructure notes (relevant to re-runs)

- Host has mpi4py; Lightning OpenMPI DDP previously hung after Using device gpu.
 - Fix: force LightningEnvironment + NCCL; disable WandB on headless hosts
 - After killed jobs, clear leftover orted processes before relaunching
 - Prefer keeping clean baseline artifacts for run **1120** unless newer runs finish successfully
-

10. Quick reference — freeze vs fresh

Run	Encoder	Freeze?
A — pretrained unfz	scmaskgit (3L early-exit)	No
B — pretrained fz	scmaskgit (3L early-exit)	Yes
C — fresh	cell (random)	No
End of brief.		